

Software Security CS-412:

Fuzzing libpng 1.2.56

Andreea-Teodora Ghinescu
EPFL

Flavien Valea
EPFL

Leïla Hammi
EPFL

Arnaud Rajon
EPFL

1 Harness Design

As a first step to start our fuzzing campaign, we began by evaluating which functions would be interesting to fuzz with our harness. Following the reference guide [1], we have considered whether to choose the decoder sequence as the entry point, versus the encoder sequence. As a first thought, the decoder seemed like the more natural choice, since its role is to process externally supplied PNG files, which aligns well with the fuzzing model where large numbers of mutated inputs are continuously provided to the target program. In contrast, the encoder mainly generates PNG output from already structured image data and therefore exposes a less input-driven attack surface. To further analyze our choice, we have consulted the `png.h` file, as well as the libpng manual for version 1.2.5 [2], more specifically the sections on Reading, Input Transformations, and Writing. Although most of the low-level transformation functions are the same between the 2 sequences, if we were to choose the encoder, our harness would need to be more specialized on crafting inputs. Consequently, we have decided to focus the campaign on the decoder.

The harness receives an input file generated by AFL++ through the command-line argument `argv[1]`. The file is opened in binary mode using `fopen`, and the resulting file pointer is later on passed to libpng through `png_init_io`. The harness initializes the main libpng structures, `png_structp png` and `png_info_ptr info`, using `png_create_read_struct` and `png_create_info_struct`. These structures maintain the decoder state and image metadata during parsing. We have placed two guards after these allocations to ensure that initialization succeeded before continuing execution. Without guarding, the functions quietly return NULL if they fail to create the structure, which later on could result in false positives for our fuzzer.

Furthermore, we implement error handling using libpng's `setjmp/longjmp` mechanism: `if (setjmp(png_jmpbuf(png)))`. We did this because malformed inputs cause libpng to do a `longjmp` on error,

which goes back to our `setjmp`. Without the `setjmp`, the program would crash or have undefined behavior, which would cause our fuzzer to detect false positives once again.

After initialization, the harness calls `png_read_info`, which parses the PNG header and metadata chunks. The harness then exercises additional library functionality related to text chunk processing. Existing text metadata is queried using `png_get_text`, and if text entries are present, they are released with `png_free_data`. A new text chunk is then inserted using `png_set_text`. We decided to explore these functions because in version 1.2.56 of libpng, there is a historical vulnerability in the function `png_set_text_2`, called by `png_set_text`. We will discuss this in section 5.

Next, the harness applies several libpng transformation functions before image decoding. These transformations normalize image representations and force the decoder through additional internal code paths. Some examples are `png_set_expand(png)` for palette expansion, or `png_set_gray_to_rgb(png)` for grayscale-to-RGB conversion. Other several transformations are conditionally enabled depending on the image properties obtained for example from `png_get_color_type`. The guards used ensure that transformations are only applied when valid for the current PNG format, in order to avoid unnecessary decoder errors but still maximize the exercised code paths.

After the transformation phase, the harness calls `png_read_update_info`, which updates the internal image information to reflect the transformations. The final image dimensions and row sizes are then retrieved using `png_get_image_height` and `png_get_rowbytes`. We use additional guards to reject images that are too large: `if (height == 0 || rowbytes == 0 || height > 4096 || rowbytes > 65536)`. These limits prevent excessive memory allocation and keep fuzzing executions efficient.

The harness then allocates memory for each image row and passes the row buffers to `png_read_image`, which performs the actual decoding of pixel data. Finally, `png_read_end` processes any remaining PNG chunks, after which all allocated memory and libpng structures are released.

2 Instrumentation and Sanitizers

During the campaign, we applied several compiler flags and configurations to instrument libpng and enable efficient vulnerability discovery. Each flag serves a specific purpose in the fuzzing workflow.

Firstly, we compiled the library with `afl-clang-fast` instead of a standard compiler. This inserts AFL’s LLVM-based instrumentation in order to record edge coverage during execution, enabling AFL to track which code paths are explored by a certain input. Without this instrumentation, AFL would operate without feedback, unable to determine if a mutation discovered new code.

We enabled `AddressSanitizer` in order to detect memory corruption bugs. ASan instruments memory operations to catch buffer overflows, use-after-free, double-free, and other memory safety violations by adding red zones around allocations and maintaining shadow memory. If omitted, memory corruption bugs would fail silently or only crash under specific conditions, causing vulnerabilities to be missed. It transforms subtle memory corruption into immediate, debuggable crashes with detailed reports.

We included debug symbols (`-g` flag) in our binary to have more readable stack traces when crashes occur. Without this flag, crash reports would only show raw memory addresses instead of function names and line numbers, making it harder to identify the cause of the crash and locate vulnerable code.

We also used optimization level 1 (`-O1` flag) to balance execution speed with instrumentation reliability. `-O0` would have resulted in slower execution, and higher optimization levels like `-O2` or `-O3` could have eliminated instrumentation points from ASan or AFL through aggressive inlining and dead code elimination.

We configured libpng to build as a static library rather than a dynamic link (`-disable-shared`). This ensures all libpng code is directly included into the harness binary, not simply referenced.

Finally, we applied AFL’s CRC removal patch (`utils/libpng_no_checksum/libpng-nocrc.patch`) so that our fuzzer can explore deep execution paths without being forever blocked by the library’s checksum.

3 Seed Corpus and Dictionary

In our setup, the seed corpus consists of PNG images in the seeds directory, starting with the PNG testcases provided with AFL++ and extended with a set of additional PNGs collected from the internet. These are PNG files that cover different aspects of the PNG format, such as varying dimensions, color types, and the presence of text or other metadata chunks. Before fuzzing, we run the seeds through `afl-cmin` to create a minimized corpus. This keeps only the seeds that contribute new coverage for the instrumented libpng, so the resulting

minimized directory is a smaller set of inputs that still spans the same control-flow paths as the original corpus.

The dictionary we used in our campaign is the PNG-specific dictionary shipped with AFL++, referenced as `png.dict`. It contains byte strings that frequently appear in PNG files, such as the PNG magic header and common chunk identifiers like IHDR, PLTE, IDAT, and IEND. During fuzzing, AFL++ uses these entries in the dictionary mutation stage: in addition to low-level bit and byte flips, it sometimes inserts or overwrites parts of the input with complete dictionary tokens. This does not make the inputs globally well-formed, but it increases the chance that important local substrings (for example, chunk names or headers) adhere to the PNG structure, resulting in libpng being able to process them further instead of failing immediately.

On the AFL++ status screen (Figure 1), separate counters report how many new execution paths were discovered by each mutation strategy, including dictionary-based mutations and the havoc/splice stages. In our instrumented run, the “fuzzing strategy yields” section showed that dictionary-based mutations discovered 18 new paths out of roughly 157k executions, while havoc/splice mutations discovered 405 new paths out of roughly 438k executions. This indicates that most of the exploration in our campaign comes from the generic havoc/splice stages, which aggressively combine multiple mutation operators, but that the dictionary still contributes a noticeable fraction of additional paths that would have been missed if mutations were purely unguided.

In more formal terms, PNG files can be viewed as elements of a language L over the byte alphabet. Libpng’s parser implements a grammar for this language, in which non-terminals describe higher-level constructs like “PNG file” or “chunk sequence” and terminals are raw byte strings such as the signature and chunk names. The seed corpus is a finite subset of L that exemplifies different derivations of this grammar. The dictionary is a finite set of terminal strings drawn from this language: fixed tokens that appear in many valid derivations. Dictionary-based mutations bias AFL++ towards generating new words that remain close to this grammar, while havoc and splice operate more randomly on whole inputs and their fragments.

4 Campaign Analysis

In [Figure 1], we can observe the following metrics obtained by our campaign after ~30 minutes:

Metric	Campaign Result
stability	100%
corpus count	768
map density	33.69%
cycles done	2

In [Figure 2], we can observe two distinct phases in the edges graph of our campaign: a rapid growth phase during

the first ~ 500 s, where edge coverage has a steep increase from ~ 855 to ~ 1000 edges, followed by the curve flattening, where only marginal gains are made, adding roughly ~ 48 new edges over the remaining ~ 1400 s. This shape suggests that the fuzzer quickly exhausted the easily reachable code paths early on but then continued to discover new edges at a slower pace.

Although the edges graph flattened after ~ 500 s, the fact that it was still finding inputs suggests that the campaign had not yet reached saturation. This is also supported by the fact that in [Figure 1], in the "pending" field of the "item geometry" section, there are still 66 inputs that have not yet gone through fuzzing. Additionally, we can observe in the same figure that the "last new find" was 14 seconds ago at the time of the screenshot, which confirms that the fuzzer was still reaching new edges. As a last argument, the "cycles done" counter shows only 2 cycles and has the color magenta (meaning it is still going through the first queue passes), whereas the AFL++ documentation [3] suggests that the fuzzer reaches saturation when the counter is blue/green.

5 Crash Triage

During our campaign, we obtained 9 saved crashes, we will focus on the first one here. First, we reproduced it using `./png_fuzz <crash_name>` which gave us the output in [Figure 3]. We can see that the crash was triggered by a write on address 0x0 (dereference of a NULL pointer) in `png_set_text_2`.

The next step is to minimize the input, in order to find the smallest input required to trigger this crash. To do so, we executed the following command: `afltmin -i <crash_name> -o ./minicrash.png - ./png_fuzz @@` which, as can be seen in [Figure 4], managed to reduce the crashing input by 65%. By using the strings command on this minimal input, we realized that it contains only 3 chunks: IHDR, IDAT, and zTXt. As zTXt is the only optional chunk and as the crash occurred in a text-related function, we assumed that this chunk is related to our crash. We then ran our binary on this minimized input using this command: `ASAN_OPTIONS=symbolize=1 ./png_fuzz minicrash.png`. But because ASan symbolization was activated by default in our binary, we obtained the same stack trace as in [Figure 3].

After examining the list of known CVEs for our version of the library, we came to the conclusion that our crash is related to CVE-2016-10087 [4], a NULL pointer dereference that occurs when all the text chunks of a png structure are freed, and then a new text chunk is set to that same structure. This happens because the `max_text` field of the png struct is not reset properly when freeing text, which causes `png_set_text_2` to believe that it can write into the text pointer of the structure when this pointer is NULL.

6 Attack Surface Analysis

Libpng is used in many real-world applications; two typical examples are web browsers such as Firefox or Chromium, and image editors such as GIMP. In a browser, libpng is used to decode PNG images that are loaded from arbitrary web pages. A realistic attack scenario is one where the attacker hosts a specially crafted PNG on a website; when the victim visits the page, the browser automatically fetches the image, passes it to libpng for decoding, and any memory safety bug in libpng's parsing or transformation code could be exploited to achieve remote code execution inside the browser process. In GIMP, libpng is used when the user opens or imports PNG images. A realistic attack scenario here is a malicious PNG attached to an email or downloaded from the internet; when the victim opens the file in GIMP to view or edit it, the application invokes libpng to parse the file, and a crafted image could trigger a memory corruption in libpng and compromise the desktop environment.

The harness used in the campaign focuses on exercising the decoding of a single PNG file all the way from the header and metadata through to the pixel data. As a result, the harness exercises much of the core image-reading code path that a browser or GUI application would hit when loading a PNG into memory for display. However, there are several important code paths in real applications that this harness does not exercise. First, it does not cover any of the writing or saving functionality of libpng, which are heavily used in GIMP and similar tools when exporting or re-saving PNG files. In GIMP, opening a malicious PNG and then re-saving it or exporting a modified version goes through libpng's encoder side; bugs in these write-side functions could be triggered only when the application writes PNGs, not when it reads them. Since the harness only creates a read struct and never calls any write-related APIs, those encoder code paths are completely out of scope for the fuzzing campaign, leaving a class of potential vulnerabilities in the saving/export pipeline unexplored.

Second, the harness runs as a simple command-line utility that reads a single PNG from disk and processes it in one shot. Real applications like Firefox and Chromium use libpng in more complex, event-driven ways, for example decoding images incrementally as data arrives over the network or integrating libpng into a larger image decoding pipeline that involves streaming, caching, downscaling and compositing. That involves libpng functions and calling patterns associated with progressive reading, incremental I/O and error recovery that the harness never touches, because it always uses `png_init_io` with a fully available file and then calls `png_read_image` to consume the entire image in one go. Those additional code paths matter for security because bugs can be input-order- or state-dependent: a vulnerability that appears only when feeding data chunk by chunk, or only when decoding interleaved image resources, might be reachable in

a browser’s network-driven path but not in the simple batch decoding path that the harness uses.

7 Binary-Only Fuzzing with QEMU Mode

To simulate a closed-source scenario, we compiled a second build of libpng 1.2.56 using plain `gcc -O1` with no AFL++ instrumentation and no sanitizers. The fuzzing harness was compiled against this vanilla library and fuzzed using AFL++ in QEMU mode (`afl-fuzz -Q`), which instruments basic blocks at runtime via QEMU user-mode emulation rather than at compile time. The CRC patch was kept in both builds so that the comparison isolates the effect of QEMU overhead rather than input rejection differences. Both campaigns used the same seed corpus and PNG dictionary, and both ran for 30 minutes [Figure 1, Figure 5].

Metric	Instrumented (ASan)	QEMU
exec/sec	427.1	1,098
edges found	204	261
corpus count	768	1,022
map density	33.69%	4.40%
saved crashes	9	20

Considering execution speed, contrary to the expectation that QEMU would always be slower, the QEMU campaign achieved approximately $2.5\times$ higher throughput (1,098 vs 427 exec/sec). This is because the instrumented build includes AddressSanitizer, which inserts shadow memory checks on every memory access and allocation. This overhead dominates the per-execution cost and outweighs the cost of QEMU’s basic block translation. In QEMU mode, each basic block is translated once on first encounter and cached, so subsequent executions pay only a modest emulation overhead with no sanitizer cost.

In terms of edges and map density, despite its higher throughput and discovering more raw edges (261 vs 204), the QEMU campaign achieved a drastically lower map density (4.40% vs 33.69%). This is explained by the difference in bitmap precision: AFL++ compile-time instrumentation assigns a unique identifier to every edge at build time, producing a precise and collision-free coverage bitmap over 3,136 known edges. QEMU mode derives coverage by XORing the translated basic block address with the previous block’s address and indexing into a fixed-size bitmap, which causes hash collisions where distinct edges map to the same entry. Crucially, without a sanitizer, QEMU mode can only detect crashes that produce a signal (SIGSEGV, SIGABRT), while the instrumented build with ASan catches subtle memory corruptions that do not immediately cause a segfault.

Regarding the corpus count, the QEMU campaign accumulated more inputs (1,022 vs 768) in the same time window, a direct consequence of its higher exec speed allowing more mutations to be tested per unit time. However, since QEMU’s

coverage bitmap is less precise, some of these corpus entries may represent inputs that exercise the same underlying code paths and would be de-duplicated under compile-time instrumentation.

8 Instrumentation Depth and Performance

The `afl-clang-fast` compilation of libpng alone reported 8,803 instrumented edges, whereas running `afl-showmap` on the final harness binary reported 3,136 edges. This difference is explained by the fact that our harness only exercises the decoder-related functions of libpng; the encoder, writer, and other unrelated code paths are omitted by the linker since the harness never references them, even though libpng is statically linked.

After 30 minutes of fuzzing in the baseline configuration [Figure 1], the campaign reached a map density of 33.69%, meaning approximately $33.69\% \times 3136 \approx 1057$ of the instrumented edges were reached. The remaining edges belong to code paths that require specific combinations of color type, bit depth, interlace mode, and ancillary chunk structure that the fuzzer had not yet synthesized from the seed corpus in this time window.

We measured execution speed in three configurations [Figure 6, Figure 7, Figure 8] using the same harness, seed corpus, and dictionary, each run for 5 minutes. The average exec/sec is computed as total executions divided by run time: $146k/303s \approx 482$, $834k/309s \approx 2,700$, and $3,800k/306s \approx 12,430$ respectively.

Configuration	avg exec/sec	Speedup
ASan + fork mode	482	$1\times$ (baseline)
No ASan + fork mode	2,700	$5.6\times$
ASan + persistent mode	12,430	$25.8\times$

Looking at ASan overhead, removing ASan while keeping fork mode yielded a $5.6\times$ speedup (482 to 2,700 exec/sec). ASan inserts shadow memory checks on every memory allocation and access, maintaining a shadow memory region that mirrors the entire process address space. This overhead accumulates significantly in a target like libpng, which performs many small allocations and pointer-based reads during chunk processing, zlib decompression, and defiltering.

Considering fork elimination, switching to persistent mode while keeping ASan enabled yielded a $25.8\times$ speedup (482 to 12,430 exec/sec). In fork mode, AFL++ calls `fork()` for every test case, duplicating the process memory mappings, resetting file descriptors, and re-initializing the fork server on each iteration. Persistent mode eliminates this by looping inside the same process up to 1,000 times before restarting via `__AFL_LOOP(1000)`, paying the fork cost only once per 1,000 executions. The minor stability drop to 99.82% in persistent mode reflects negligible state accumulation between iterations, which does not affect the validity of the campaign.

9 Appendix

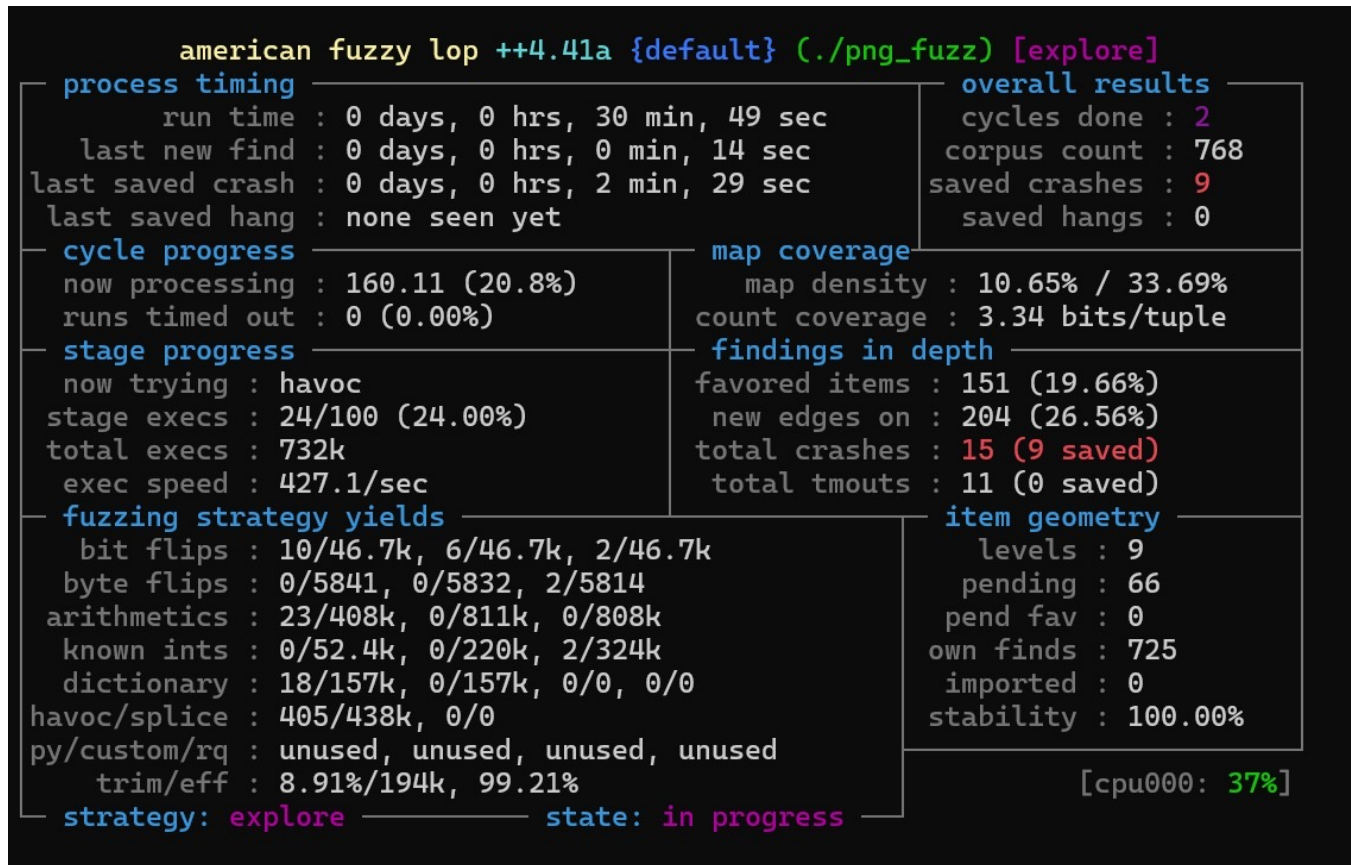


Figure 1: AFL++ status screen after 30 min. - Instrumented.

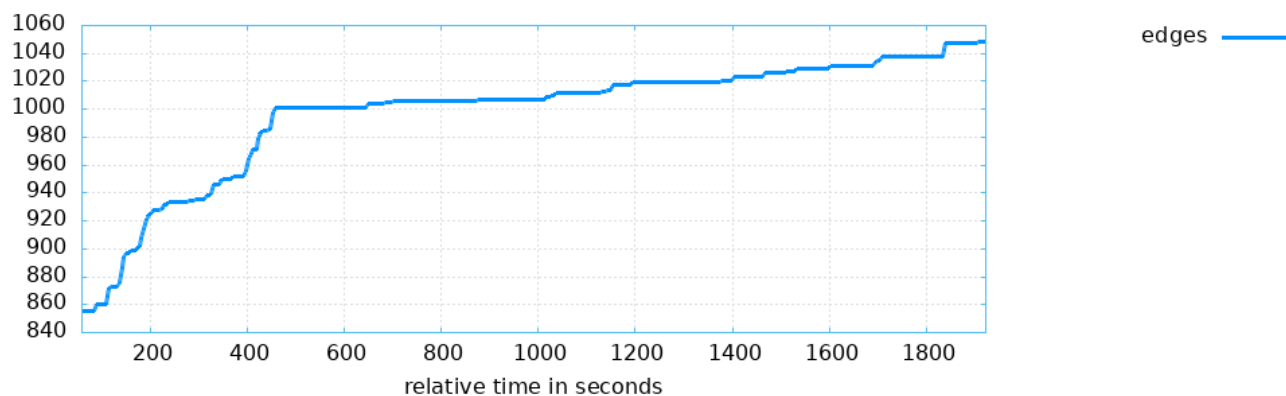


Figure 2: Edges graph.

```
[AFL++ d311f6d5bd2b] /home/softsec/work # ./png_fuzz findings/default/crashes/id\:000000\,sig\:11\,src\:000003\,time\:157501\,execs\:76122\,op\:havoc\,rep\:2
NOTE: CRC in the file is 0x31e8965f, change to 0xd96aa512
libpng warning: Unknown compression type in zTXt chunk
libpng warning: Data error in compressed datastream in zTXt chunk
AddressSanitizer:DEADLYSIGNAL
=====
==2453==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000 (pc 0x55dc61332f0b bp 0x515000000344 sp 0x7ffedf63abe0 T0)
==2453==The signal is caused by a WRITE memory access.
==2453==Hint: address points to the zero page.
#0 0x55dc61332f0b in png_set_text_2 /home/softsec/libpng-1.2.56/pngset.c
#1 0x55dc61332acb in png_set_text /home/softsec/libpng-1.2.56/pngset.c:654:10
#2 0x55dc6132abdb in main /home/softsec/work/./src/harness.c:59:5
#3 0x7f9e120af1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#4 0x7f9e120af28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
#5 0x55dc612494c4 in _start (/home/softsec/work/png_fuzz+0x314c4) (BuildId: 240d891ab40e7a0fb11cd1a11b046612d45b853e)

==2453==Register values:
rax = 0x0000000000000003  rbx = 0x0000000000000000  rcx = 0x0000000000000000  rdx = 0x0000000000000000
rdi = 0x0000000000000000  rsi = 0x0000000000000004  rbp = 0x0000515000000344  rsp = 0x00007ffedf63abe0
r8 = 0x00000abb8c273c64  r9 = 0x00000abc0c26bc01  r10 = 0xf9f9f904f9f9f905  r11 = 0x906eddb766a74b01
r12 = 0x0000000000000004  r13 = 0x00007ffedf63ad68  r14 = 0x0000ffffdbec75ad  r15 = 0x0000000000000000
AddressSanitizer can not provide additional info.
SUMMARY: AddressSanitizer: SEGV /home/softsec/libpng-1.2.56/pngset.c in png_set_text_2
==2453==ABORTING
```

Figure 3: Crash reproduction.

```
[AFL++ d311f6d5bd2b] /home/softsec/work # afl-tmin -i findings/default/crashes/id\:000000\,sig\:11\,src\:000003\,time\:157501\,execs\:76122\,op\:havoc\,rep\:2 -o ./mini-crash.png -- ./png_fuzz @@
afl-tmin++4.41a by Michal Zalewski

[+] Read 164 bytes from 'findings/default/crashes/id\:000000,sig\:11,src\:000003,time\:157501,execs\:76122,op\:havoc,rep\:2'.
[*] Performing dry run (mem limit = 0 MB, timeout = 1000 ms)...
[+] Program exits with a signal, minimizing in crash mode.
[*] Stage #0: One-time block normalization...
[+] Block normalization complete, 112 bytes replaced.
[*] --- Pass #1 ---
[*] Stage #1: Removing blocks of data...
Block length = 16, remaining size = 164
Block length = 8, remaining size = 64
Block length = 4, remaining size = 64
Block length = 2, remaining size = 60
Block length = 1, remaining size = 58
[+] Block removal complete, 107 bytes deleted.
[*] Stage #2: Minimizing symbols (24 code points)...
[+] Symbol minimization finished, 4 symbols (6 bytes) replaced.
[*] Stage #3: Character minimization...
[+] Character minimization done, 5 bytes replaced.
[*] --- Pass #2 ---
[*] Stage #1: Removing blocks of data...
Block length = 4, remaining size = 57
Block length = 2, remaining size = 57
Block length = 1, remaining size = 57
[+] Block removal complete, 0 bytes deleted.

File size reduced by : 65.24% (to 57 bytes)
Characters simplified : 215.79%
Number of execs done : 279
Fruitless execs : path=231 crash=0 hang=0

[*] Writing output to './mini-crash.png'...
[+] We're done here. Have a nice day!
```

Figure 4: Crash input minimization.

```
american fuzzy lop ++4.41a {default} (./png_fuzz_qemu) [explore]
┌ process timing ────────────────────────────────────────────┐ overall results ────────────────────────────────────────────┐
│ run time : 0 days, 0 hrs, 31 min, 5 sec                    │ cycles done : 7                                                    │
│ last new find : 0 days, 0 hrs, 0 min, 10 sec               │ corpus count : 1022                                                │
│ last saved crash : 0 days, 0 hrs, 0 min, 28 sec           │ saved crashes : 20                                                 │
│ last saved hang : none seen yet                            │ saved hangs : 0                                                    │
└──────────────────────────────────────────────────────────────┘──────────────────────────────────────────────────────────────┘
┌ cycle progress ────────────────────────────────────────────┐ map coverage ────────────────────────────────────────────┐
│ now processing : 856.21 (83.8%)                             │ map density : 1.26% / 4.40%                                       │
│ runs timed out : 0 (0.00%)                                 │ count coverage : 3.11 bits/tuple                                  │
└──────────────────────────────────────────────────────────────┘──────────────────────────────────────────────────────────────┘
┌ stage progress ────────────────────────────────────────────┐ findings in depth ────────────────────────────────────────────┐
│ now trying : havoc                                          │ favored items : 181 (17.71%)                                       │
│ stage execs : 198/300 (66.00%)                             │ new edges on : 261 (25.54%)                                       │
│ total execs : 1.84M                                         │ total crashes : 41 (20 saved)                                     │
│ exec speed : 1098/sec                                       │ total tmouts : 0 (0 saved)                                        │
└──────────────────────────────────────────────────────────────┘──────────────────────────────────────────────────────────────┘
┌ fuzzing strategy yields ───────────────────────────────────┐ item geometry ────────────────────────────────────────────┐
│ bit flips : 9/74.1k, 15/74.1k, 3/74.1k                   │ levels : 9                                                         │
│ byte flips : 0/9262, 0/9250, 1/9226                      │ pending : 8                                                        │
│ arithmetics : 23/647k, 0/1.29M, 0/1.28M                  │ pend fav : 0                                                       │
│ known ints : 0/83.1k, 2/350k, 4/515k                     │ own finds : 977                                                    │
│ dictionary : 1/250k, 0/250k, 0/0, 0/0                   │ imported : 0                                                        │
│ havoc/splice : 598/1.32M, 0/0                             │ stability : 100.00%                                               │
│ py/custom/rq : unused, unused, unused, unused            │                                                                    │
│ trim/eff : 10.55%/326k, 99.38%                           │                                                                    │
└──────────────────────────────────────────────────────────────┘──────────────────────────────────────────────────────────────┘
strategy: explore state: in progress [cpu000: 25%]
```

Figure 5: AFL++ status screen - QEMU 30 min.

```

american fuzzy lop ++4.41a {default} (./png_fuzz) [explore]
process timing
  run time : 0 days, 0 hrs, 5 min, 3 sec
  last new find : 0 days, 0 hrs, 0 min, 1 sec
  last saved crash : 0 days, 0 hrs, 1 min, 38 sec
  last saved hang : none seen yet
cycle progress
  now processing : 18.0 (4.3%)
  runs timed out : 0 (0.00%)
stage progress
  now trying : trim 4/4
  stage execs : 7/12 (58.33%)
  total execs : 146k
  exec speed : 415.7/sec
fuzzing strategy yields
  bit flips : 4/4144, 2/4142, 1/4138
  byte flips : 0/518, 0/516, 2/512
  arithmetics : 5/36.1k, 0/70.9k, 0/70.1k
  known ints : 0/4598, 0/19.4k, 1/28.4k
  dictionary : 12/14.0k, 0/14.0k, 0/0, 0/0
  havoc/splice : 188/125k, 0/0
  py/custom/rq : unused, unused, unused
  trim/eff : 0.93%/4886, 97.49%
overall results
  cycles done : 0
  corpus count : 420
  saved crashes : 2
  saved hangs : 0
map coverage
  map density : 7.63% / 30.41%
  count coverage : 2.92 bits/tuple
findings in depth
  favored items : 132 (31.43%)
  new edges on : 157 (37.38%)
  total crashes : 2 (2 saved)
  total tmouts : 0 (0 saved)
item geometry
  levels : 2
  pending : 407
  pend fav : 130
  own finds : 377
  imported : 0
  stability : 100.00%
[cpu000: 25%]
strategy: explore state: in progress

```

Figure 6: AFL++ status screen - ASan + fork mode.

```

american fuzzy lop ++4.41a {default} (./png_fuzz_nosan) [explore]
  process timing                               overall results
    run time : 0 days, 0 hrs, 5 min, 9 sec      cycles done : 4
    last new find : 0 days, 0 hrs, 0 min, 8 sec  corpus count : 698
  last saved crash : 0 days, 0 hrs, 0 min, 24 sec saved crashes : 19
    last saved hang : none seen yet              saved hangs : 0
  cycle progress
    now processing : 158.19 (22.6%)
    runs timed out : 0 (0.00%)
  stage progress
    now trying : havoc
    stage execs : 57/100 (57.00%)
    total execs : 834k
    exec speed : 2831/sec
  fuzzing strategy yields
    bit flips : 23/30.7k, 10/30.7k, 6/30.7k
    byte flips : 0/3840, 1/3824, 1/3792
    arithmetics : 34/268k, 1/529k, 0/524k
    known ints : 0/34.4k, 1/144k, 0/211k
    dictionary : 2/103k, 0/104k, 0/0, 0/0
    havoc/splice : 442/659k, 0/0
    py/custom/rq : unused, unused, unused, unused
    trim/eff : 23.97%/66.9k, 98.67%
  map coverage
    map density : 6.51% / 31.50%
    count coverage : 3.33 bits/tuple
  findings in depth
    favored items : 134 (19.20%)
    new edges on : 184 (26.36%)
    total crashes : 41 (19 saved)
    total tmtouts : 0 (0 saved)
  item geometry
    levels : 17
    pending : 68
    pend fav : 0
    own finds : 694
    imported : 0
    stability : 100.00%
[cpu000: 50%]
strategy: explore state: in progress

```

Figure 7: AFL++ status screen - No ASan + fork mode.

american fuzzy lop ++4.41a {default} (./png_fuzz_persistent) [explore]	
process timing run time : 0 days, 0 hrs, 5 min, 6 sec last new find : 0 days, 0 hrs, 0 min, 12 sec last saved crash : 0 days, 0 hrs, 1 min, 6 sec last saved hang : none seen yet	overall results cycles done : 17 corpus count : 928 saved crashes : 26 saved hangs : 0
cycle progress now processing : 502.70 (54.1%) runs timed out : 0 (0.00%)	map coverage map density : 10.76% / 34.91% count coverage : 3.24 bits/tuple
stage progress now trying : havoc stage execs : 299/300 (99.67%) total execs : 3.80M exec speed : 12.0k/sec	findings in depth favored items : 173 (18.64%) new edges on : 239 (25.75%) total crashes : 113 (26 saved) total tmouts : 11 (0 saved)
fuzzing strategy yields bit flips : 24/36.8k, 8/36.7k, 7/36.7k byte flips : 0/4595, 0/4577, 1/4541 arithmetics : 36/320k, 1/633k, 0/627k known ints : 4/41.2k, 2/173k, 2/253k dictionary : 7/124k, 0/124k, 0/0, 0/0 havoc/splice : 643/3.33M, 0/0 py/custom/rq : unused, unused, unused trim/eff : 16.56%/114k, 98.69%	item geometry levels : 30 pending : 0 pend fav : 0 own finds : 924 imported : 0 stability : 99.82%
strategy: explore state: in progress [cpu000: 37%]	

Figure 8: AFL++ status screen - ASan + persistent mode.

References

- [1] Course Staff. Software security cs-412: Fuzzing libpng, 2026. Course handout.
- [2] Glenn Randers-Pehrson. libpng 1.2.5 manual. <https://www.libpng.org/pub/png/libpng-1.2.5-manual.html>, 2002.
- [3] AFL++ Project. The afl-fuzz approach. https://aflplus.plus/docs/afl-fuzz_approach/.
- [4] National Vulnerability Database. CVE-2016-10087. <https://nvd.nist.gov/vuln/detail/CVE-2016-10087>, 2016.